

# Git 버전 관리 전략 — Link/Tower 2 프로젝트 병행 운용

- 작성일: 2026-04-16
- 대상: `apps/system` (Link), `apps/loraPing_tower` (Tower)
- 목적: 두 프로젝트를 동시에 진행하면서, 검증된 이전 상태로 안전하게 복원할 수 있는 체계 수립

## 1. 배경

### 1.1 현재 상황

| 항목          | 상태                                                                                                                |
|-------------|-------------------------------------------------------------------------------------------------------------------|
| 저장소         | <code>revita</code> (단일 repo, 두 프로젝트 동거)                                                                          |
| 브랜치         | <code>main</code> 단일                                                                                              |
| 마지막 커밋      | <code>3b26307c</code> (04-15 TC-06)                                                                               |
| uncommitted | TC-07 관련 변경 ( <code>lora_module.c</code> , <code>loraPing_tower/main.c</code> , <code>lora_byte_proto.h</code> 등) |

### 1.2 문제

- 두 프로젝트(Link/Tower)를 동시에 수정하다가 문제가 생기면, 어느 시점으로 돌아가야 하는지 불분명
- 커밋 메시지만으로는 "이 시점이 정상 동작하는 상태였는지" 판별 어려움
- 실험적 변경이 main에 바로 섞이면 안전한 기준선이 없어짐

## 2. 전략: Tag + Branch 조합

### 2.1 Git Tag — 검증된 시점 마킹

TC 시험 PASS 후, 해당 커밋에 태그를 찍는다.

태그는 "이 시점은 검증 완료된 안전한 상태" 라는 의미다.

```
# 태그 생성 (현재 커밋에)
git tag tc06-pass

# 특정 커밋에 태그
git tag tc06-pass 3b26307c

# 설명 포함 태그 (annotated tag)
git tag -a tc06-pass -m "TC-01~TC-06 전항 PASS, ACK 원샷 타이머 + cmd큐 재arm"

# 태그 목록 확인
git tag -l

# 태그 시점의 코드 확인 (읽기 전용)
git checkout tc06-pass
```

```
# 다시 최신으로 복귀
git checkout main
```

## 태그 네이밍 규칙

```
tc{번호}-pass      # TC 시험 통과 시점
phase{번호}-done   # Phase 구현 완료 시점
v{버전}            # 릴리스 버전
{날짜}-stable      # 날짜 기준 안정 빌드 (예: 0415-stable)
```

## 예시 타임라인

```
3b26307c ← tc06-pass (현재)
|
├─ TC-07 커밋 ← tc07-pass
|
├─ Phase 1 완료 ← phase1-done
|
└─ ...
```

## 2.2 Git Branch — 실험적 작업 분리

새로운 기능이나 큰 변경을 시도할 때, **main**에서 브랜치를 만든다.

```
# 새 브랜치 생성 + 전환
git checkout -b feature/phase1-codec

# 작업 진행 (여러 커밋 가능)
git add ...
git commit -m "Phase 1: type_code enum 정의"
git commit -m "Phase 1: CREATE 인코드/디코드"

# 잘 되면 → main에 합치기
git checkout main
git merge feature/phase1-codec

# 실패하면 → 브랜치 버리고 main으로 복귀
git checkout main
git branch -D feature/phase1-codec    # 로컬 브랜치 삭제
```

## 브랜치 네이밍 규칙

```
feature/{기능명}      # 새 기능 개발 (예: feature/phase1-codec)
fix/{버그명}          # 버그 수정 (예: fix/rx-init-error)
test/{시험명}         # 시험용 임시 변경 (예: test/cmd-queue-stress)
```

### 3. 운용 시나리오

#### 시나리오 A: TC 시험 후 안정 버전 확보

1. TC-07 코드 작성
2. 빌드 + 플래시 + cutecom으로 시험
3. PASS 확인
4. git add + git commit
5. git tag tc07-pass ← ★ 안전 마킹

#### 시나리오 B: 실험적 변경 후 실패 → 복원

1. git checkout -b test/rx-duty-cycle ← 브랜치 분리
2. SX1262 RX 듀티 사이클 코드 추가
3. 빌드 + 시험 → 실패
4. git checkout main ← main으로 복귀 (변경 사라짐)
5. git branch -D test/rx-duty-cycle ← 브랜치 삭제

#### 시나리오 C: 특정 시점으로 되돌리기

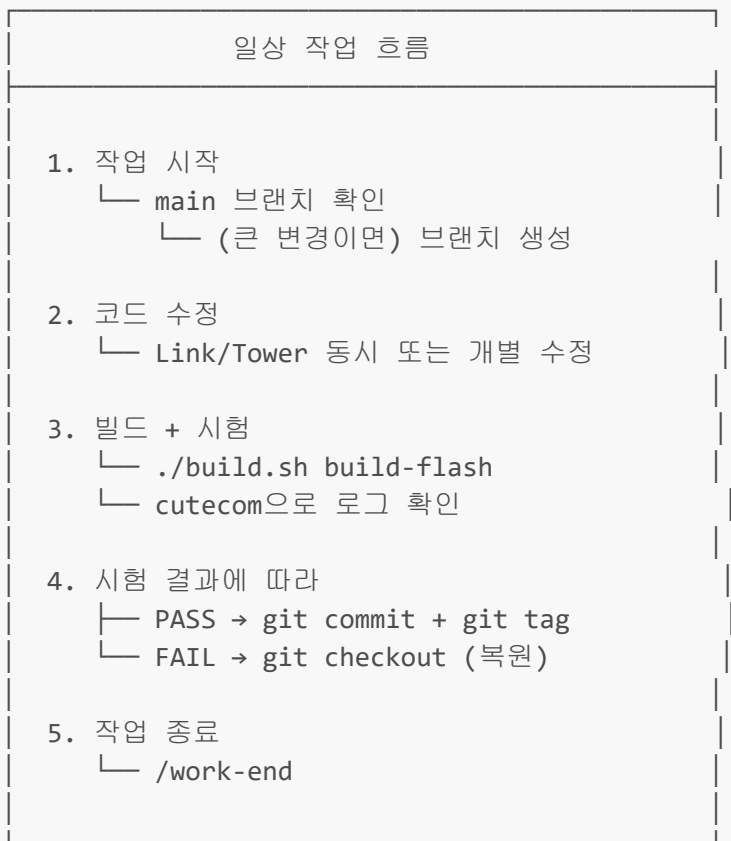
```
# TC-06 시점의 코드로 빌드하고 싶을 때
git stash                # 현재 uncommitted 변경 임시 저장
git checkout tc06-pass   # TC-06 시점으로 이동
./build.sh              # 빌드 + 플래시
git checkout main        # 최신으로 복귀
git stash pop            # 임시 저장 복원
```

#### 시나리오 D: 두 프로젝트 중 하나만 되돌리기

```
# Tower만 TC-06 시점으로 되돌리기 (Link는 유지)
git checkout tc06-pass -- apps/loraPing_tower/src/main.c

# 확인 후 문제 없으면 커밋
git add apps/loraPing_tower/src/main.c
git commit -m "Tower: TC-06 시점으로 복원"
```

## 4. 일상 워크플로우 요약



## 5. 유용한 명령어 모음

### 상태 확인

```
git status          # 현재 변경사항 확인
git log --oneline -10 # 최근 커밋 10개
git tag -l          # 태그 목록
git branch -a        # 브랜치 목록
git diff             # uncommitted 변경사항
git diff tc06-pass..HEAD # 특정 태그 이후 변경사항
```

### 태그 관리

```
git tag tc07-pass          # 현재 커밋에 태그
git tag -a tc07-pass -m "TC-07 PASS: module_type 라우팅" # 설명 포함
git tag -d tc07-pass       # 태그 삭제
git show tc06-pass         # 태그 시점 커밋 정보
```

### 복원

```
git stash                # 현재 변경 임시 저장
git stash pop            # 임시 저장 복원
git stash list           # 임시 저장 목록
git checkout <tag> -- <file> # 특정 파일만 특정 시점으로
git checkout main        # main으로 복귀
```

## 브랜치

```
git checkout -b <name>    # 브랜치 생성 + 전환
git checkout main         # main으로 전환
git merge <branch>        # 브랜치 합치기
git branch -D <branch>    # 브랜치 삭제
```

## 6. 즉시 적용 사항

현재 상태에서 바로 적용할 것:

```
# 1. TC-06 커밋에 태그 찍기
git tag -a tc06-pass 3b26307c -m "TC-01~TC-06 PASS: 비동기RX + ACK원샷타이머 + cmd
큐재arm"

# 2. uncommitted TC-07 변경사항 커밋
git add zephyr_workspace/apps/system/src/lora_module.c
git add zephyr_workspace/apps/system/src/lora_module.h
git add zephyr_workspace/apps/loraPing_tower/src/main.c
git add zephyr_workspace/apps/lora_byte_proto.h
git add zephyr_workspace/apps/system/src/telemetry_task.c
git commit -m "TC-07: module_type 라우팅 + packet_id 흐름 정리"

# 3. TC-07 태그
git tag -a tc07-pass -m "TC-07 PASS: module_type별 RX 라우팅 골격, ACK suppressed
정책"
```

작성: Claude Code